

# 오늘의 작업기록 — 2026년 7월 2일

## mini-ledger Layer 1 — 환불(refund) 구현

(환불 = 원본 찾기 + 검증 + 반대 거래 + 기록 — 5단계 풀 사이클 첫 도달. 단 이날은 구현까지, 복기는 다음날로)

### 한눈에

| 항목     | 내용                                                                               |
|--------|----------------------------------------------------------------------------------|
| 목표     | Layer 1 환불 — TRANSFER 거래를 되돌리기                                                   |
| 결과     | 완료 ✓ — 완료 기준 3개 전부 통과                                                            |
| 커밋     | <code>add refund for transfers</code> ( <code>85b3af6</code> , 4 files, +94/-41) |
| 저장소    | github.com/Taewoo-Won/mini-ledger                                                |
| 오늘의 핵심 | 설계 결정 4개 확정 + 첫 시그니처 변경 연쇄 수정                                                    |
| 새 개념   | 거래 id(자동 카운터) · REFUND enum · 원본 불변(되돌림도 새 거래) · 이중환불=역등성 재등장                    |
| 운영 방식  | ★ 이날 한정 예외 — 잔디 우선이라 코드 먼저 완성→커밋, 이해(복기)는 7/3로 분리                                |
| 왜 예외   | 같은 날 SQLD CBT도 만들어야 해 세션 빠듯 → 잔디부터 확보. 환불은 배운 것들의 재조합이라 역방향 복기로 충분               |

원칙적으로 mini-ledger는 "내가 먼저 짜고 Claude가 봐주는" 솔로 런이다. 이날은 여건상 반대로 — 코드를 먼저 받아 잔디를 확보하고, 이해는 다음날(7/3) 복기로 넘겼다. 이 역순은 이날 한정 예외이며, 다음 기능(정산·복식부기)은 다시 솔로 런으로 돌아간다. 정직하게 남긴다.

### 0. 출발점 / 오늘의 목표

- **이전까지:** 증분 1~5 + 조회(계좌·금액)까지. Layer 1: 감사로그✓ 역등성✓ 조회✓.
- **오늘 목표:** 환불 — 잘못된/취소된 TRANSFER를 되돌린다.
- **왜 이것:** 결제에서 환불은 필수 경로. 그리고 어제 만든 조회가 정확히 이걸 위한 온램프였다 — 환불 = 원본 찾기(조회) + 반대 거래.
- **스코프:** TRANSFER 환불만. (DEPOSIT/WITHDRAW 환불까지 하면 스코프가 불고, 핵심 개념은 TRANSFER 하나로 다 배움.)
- **다음:** 정산 · 복식부기 (Layer 1 나머지) — 솔로 런으로.
- **도구:** 폰/VPS EOF 복불 (컴 안 거침). 잔디 우선.

## PART 1 — 도메인: 되돌림도 새 거래다

결제 시스템의 대원칙 하나가 여기 박힌다:

원본 기록은 절대 건드리지 않는다. 되돌림도 새 거래다.

통장에서 잘못된 이체를 지우는 게 아니라, 반대 방향 이체가 한 줄 추가되는 것. 6/27~28에 만든 append-only 감사 로그가 그래서 산다. 환불 = 원본 찾기 + 반대 거래 생성. → 그래서 환불 후 history엔 원본(TRANSFER)과 환불(REFUND)이 둘 다 남는다. 원본은 불변. 이게 "1년 뒤 기록만 보고 재구성 가능"의 핵심.

## PART 2 — 설계 결정 4개 (오늘의 본체)

환불은 조회(읽기)와 달리 찾고+검증하고+상태 바꾸고+기록하는 풀 사이클. 5단계 전 칸이 차고, 결정이 4개였다.

| 결정         | 질문                | 답                    | 어느 파일                                         |
|------------|-------------------|----------------------|-----------------------------------------------|
| 1. 원본 지목   | "어떤 거래?"를 어떻게 특정? | 거래마다 id(자동 카운터)      | Transaction(필드) + Bank(카운터) + findTransaction |
| 2. 환불 type | 반대 거래를 뭐로 기록?     | REFUND enum 신설       | TransactionType                               |
| 3. 검증      | 없는 거래? 이중 환불?     | 예외 + Set(역등성 패턴)     | Bank.refund                                   |
| 4. 돈 되돌리기  | 잔액 어떻게?           | 원본 반대로 이동(기존 도구 재사용) | Bank.refund                                   |

### 2-1. 결정 1 — 원본 지목 (★ 오늘 제일 큰 결정)

계좌를 지목할 때 id("a")를 썼듯, 거래도 지목하려면 id가 필요하다. 지금까지진 거래에 번호가 없었다 → 추가.

- Transaction에 long id 필드 추가.
- Bank에 nextTxId 카운터 — 거래 만들 때마다 nextTxId++ 로 1,2,3... 자동 부여.
- findTransaction(txId) — 그 번호로 원본 찾기 (필터링 공식과 같은 순회 검색).

→ ★ 이게 첫 "시그니처 변경 연쇄"다. Transaction 생성자가 (id, type, from, to, amount) 로 바뀌니, Bank에서 Transaction을 만드는 모든 곳 (deposit/withdraw/transfer)이 전부 수정돼야 했다. 6/25에 배운 "시그니처 변경 → 호출처 다 에러 → 에러가 다음 고칠 곳"의 실전. 이번엔 컴파일 에러 대신 미리 다 반영.

## 2-2. 결정 2 — REFUND enum

TRANSFER b→a로 기록하면 1년 뒤 "환불"인지 "그냥 역방향 이체"인지 구분 안 된다. → REFUND를 TransactionType에 신설. 기록에 "이건 환불"이라고 명확히 남는다. (6/27 "기록만으로 재구성" 원칙.)

## 2-3. 결정 3 — 이중환불 방지 = 역등성 재등장

같은 거래를 두 번 환불하면 돈이 두 번 돌아간다 = 이중 이체만큼 위험한 사고. 6/29 역등성과 똑같은 패턴: - Set<Long> refundedIds — 환불된 원본 id 저장. - refund 들어 오면 refundedIds.contains(txId) 로 검사 → 이미 있으면 return (스킵). - 환불 성공 후 refundedIds.add(txId) . → usedKeys(역등성)와 판박이. "검사는 위, 저장은 성공 후."

## 2-4. 결정 4 — 돈 되돌리기 (기존 도구 재사용)

TRANSFER a→b 1000의 환불 = b에서 1000 빼서 a에게 1000. 새 도구 안 만든다 — 이미 있는 걸 반대로:

```
toAccount.withdraw(original.getAmount()); // 받았던 b에서 빼고
fromAccount.deposit(original.getAmount()); // 보냈던 a에게 돌려줌
```

→ Account의 withdraw/deposit 재사용. 방향만 반대.

# PART 3 — refund 메서드 (검증→실행→기록)

```
public void refund(long txId) {
    Transaction original = findTransaction(txId); // 원본 찾기
    if (original == null) { // [검증] 없는 거래
        throw new IllegalArgumentException("환불할 거래를 찾을 수 없습니다: " + txId);
    }
    if (refundedIds.contains(txId)) { // [검증] 이중환불
        return;
    }
    Account fromAccount = getAccount(original.getFrom());
    Account toAccount = getAccount(original.getTo());
    toAccount.withdraw(original.getAmount()); // [실행] 반대로 이동
    fromAccount.deposit(original.getAmount());
    history.add(new Transaction(nextTxId++, TransactionType.REFUND,
        original.getTo(), original.getFrom(), original.getAmount())); // [기록] REFUND
    refundedIds.add(txId); // 환불됨 표시
}
```

→ 6/29에 세운 "검증(위) → 실행(가운데) → 기록(아래)" 순서 그대로. 그리고 이게 전부 어제까지 배운 것의 재조합이다. 원본 찾기(필터링/조회) + 이중환불 막기(역등성 Set) + 돈 이동(transfer 로직) + 기록(history.add).

## refund가 txId 하나만 받는 이유

원본을 찾으면 그 안에 from·to·amount가 다 들어있다. 그래서 "몇 번 거래?"(txId)만 받으면 나머지는 원본에서 꺼내 쓴다. 매개변수 최소화.

# PART 4 — 검증 (완료 기준 3개, Main 실행)

```
환불 전: a=1100, b=4500 (a가 1000 보냄)
환불 후: a=2100, b=3500 ← 원위치 ✓

거래내역(환불 후):
1 TRANSFER a -> b : 1000 ← 원본 그대로 (안 지워짐) ✓
2 REFUND b -> a : 1000 ← 환불이 새 거래로 추가 ✓

이중환불 시도: a=2100, b=3500 ← 안 바뀜(차단) ✓
없는 거래(999): 잡았다: 환불할 거래를 찾을 수 없습니다: 999 ← 예외 ✓
```

완료 기준 3개 전부 통과: 1. 정상 환불 → 두 잔액 원위치 + history에 원본·환불 둘 다(원본 불변)✓ 2. 같은 거래 두 번 환불 → 두 번째 차단✓ 3. 없는 거래 환불 → 예외✓

## 5. 산출물

- 파일 4개 변경:
- TransactionType.java — REFUND 추가

- `Transaction.java` — id 필드 + 생성자 변경
- `Bank.java` — nextTxId 카운터, refundedIds Set, findTransaction, refund
- `Main.java` — 환불 테스트(완료 기준 3개)
- 커밋: `85b3af6` "add refund for transfers" (4 files, +94/-41). `44f09cf`. `85b3af6` .
- 커밋 계보: `d82e58e` (역등성) → `ee9f03b` (계좌조회) → `44f09cf` (금액조회) → `85b3af6` (환불)

## 6. 다음 작업

- 7/3(다음날): 이 환불 코드 복기 — 완성본을 역방향으로 해체. 4개 결정이 코드에 어떻게 박혔는지, 재조합 구조, "왜?" 청산.
- 정산 — 이체에 수수료 분배 (솔로 런 복귀).
- 복식부기 — 모든 거래가 양쪽 계정을 건드리게 (Layer 1 마지막, 좀 무거움).

## 7. 개념 사전

- 거래 id / 자동 카운터: 각 거래에 고유 번호. Bank가 `nextTxId` 를 들고 `++` 로 부여. "이 거래" 지목의 근거.
- REFUND (enum 값): 환불 거래임을 기록에 명시. 역방향 이체와 구분.
- 원본 불변 (append-only): 되돌림은 원본을 지우지 않고 반대 거래를 추가. 감사 로그의 핵심.
- 이중환불 방지 (Set): 환불된 원본 id를 Set에 저장, 재환불 시 차단. 역등성과 동일 패턴.
- 시그니처 변경 연쇄: 생성자 인자가 바뀌면 그걸 호출하는 모든 곳이 수정돼야 함. (Transaction id 추가 → Bank 3곳.)
- 매개변수 최소화: 원본에 정보가 다 있으면 호출자는 식별자(txid)만 넘긴다.
- findTransaction: id로 원본을 찾는 순회 검색. 필터링 공식과 같은 뼈대(순회+조건).

## 8. 비유 모음

- 환불 = 회계장부 정정: 틀린 줄을 지우개로 지우지 않는다(감사 불가). 대신 "위 항목 취소" 줄을 새로 쓴다. 원본과 취소가 나란히 남아야 추적된다.
- 거래 id = 영수증 번호: 계좌에 이름(id)이 있듯, 영수증(거래)에도 일련번호. "3번 영수증 환불"이 가능해진다.
- REFUND type = 도장 색: 같은 "돈 이동"이라도 환불엔 빨간 "REFUND" 도장. 나중에 색만 봐도 구분.
- refundedIds = 환불 도장 찍은 목록: 한 번 환불한 영수증 번호를 적어두고, 또 들어오면 "이미 처리됨" 반려.

## 9. 검증 질문 (7/3 복기 때 답할 것 — 답 적지 말 것)

1. "원본을 지목한다"를 위해 3개 파일에 각각 뭘 추가했나? (Transaction / Bank 필드 / Bank 메서드)
2. 환불을 TRANSFER b→a가 아니라 REFUND로 기록하는 이유는? 어떤 원칙과 연결되나?
3. 이중환불 방지는 6/29의 무엇과 같은 패턴인가? 검사와 저장의 위치는?
4. refund가 txid 하나만 받아도 되는 이유는? from/to/amount는 어디서 오나?
5. Transaction에 id를 추가했을 때 Bank의 어디어디가 바뀌어야 했나? 왜?
6. 돈을 되돌릴 때 새 메서드를 안 만들고 뭘 재사용했나?

## 10. 회고

환불은 지금까지 만든 것 중 가장 종합적이었다. 조회는 읽기뿐이었지만, 환불은 찾고(조회) → 검증하고(역등성) → 상태 바꾸고(이체) → 기록하는(감사로그) 풀 사이클이다. 놀라운 건, 이 네 조각이 전부 6월에 이미 배운 것들이라는 점이다. 새 개념은 "거래 id"와 "REFUND enum" 정도고, 나머지는 재조합이었다. Layer 1을 쌓아온 게 여기서 복리로 돌아왔다.

다만 이날은 정직하게 밝힌다 — **여건상 야매로 갔다**. SQLD CBT도 만들어야 해 세션이 빠듯했고, 잔디를 먼저 확보해야 했다. 그래서 평소 원칙(내가 먼저 짜기)과 반대로, 코드를 먼저 받아 커밋하고 이해는 7/3로 미뤘다. 이걸 이날 한정 예외다. 환불이 "재조합"이라 역방향 복기로 충분하다는 판단이 있었기에 택한 거고, 다음 기능(정산·복식부기)은 다시 솔로 런으로 돌아간다. 편의가 원칙을 잡아먹지 않게, 예외는 예외로 기록해둔다.

그래도 완료 기준 3개를 눈으로 확인하고 커밋한 건 지켰다 — 베끼고 넘어간 게 아니라, 최소한 "작동하는지"는 검증했다. 진짜 이해는 내일 채운다.

**한 줄:** 환불로 5단계 풀 사이클에 처음 도달했고, 그 조각들이 전부 6월에 배운 것의 재조합임을 봤다 — 단 이날은 잔디 우선의 야매였음을 정직하게 남기고, 복기는 7/3로.